

Final Test

2026

Contenido

Intellectual Property and Permitted Use	3
Python Data Analysis Exam	4
E-commerce Order Records Audit (Simplified Version)	4
Instructions	4
Grading Table (Rubric)	5

Intellectual Property and Permitted Use

This course is the intellectual property of Tania Rodríguez Suárez. Its reproduction, distribution, or sale without prior authorization is strictly prohibited. This material is exclusively for personal and educational use by those who have legitimately acquired it.

Python Data Analysis Exam

E-commerce Order Records Audit

(Simplified Version)

An e-commerce company exports daily **order records** as text strings. These records are inconsistent and must be cleaned, validated and analyzed to create a basic sales report.

You will progressively transform raw text data into useful business information.

Instructions

read carefully:

- You must work in **ONE single Python script**.
- The script is divided into **10 sequential objectives**.
(You **cannot** solve Objective N without Objective N-1.)
- **Do NOT use:** try, except, or anything related to exception handling.
- Use only Python tools taught in the PDFs:
 - Strings: lower(), upper(), strip(), replace(), count(), startswith(), endswith(), split()
 - Lists: len(), append(), remove(), pop(), count(), sort()
 - Loops: for, while, range, break, continue
 - Functions with def and return
 - Validations with if / elif / else

Grading Table (Rubric)

Objective	What is evaluated	What will be reviewed (variables / structures)	Points
1	Raw dataset realism and setup	• A list containing raw records (e.g. raw_orders) • Use of constants: VALID_STATUSES, HIGH_VALUE_THRESHOLD, MIN_VALID_AMOUNT • Correct use of len() to show number of raw records	8
2	Cleaning logic and reuse via function	• Existence of a cleaning function (e.g. clean_record) • A new list with cleaned records (e.g. cleaned_orders) • Correct string transformations applied before validation	10
3	Validation and data separation	• Lists storing valid data (e.g. order_ids, emails, amounts, statuses) • A list for invalid records (e.g. invalid_records) • Conditional validations and correct use of continue	14
4	Filtering completed orders	• A list storing amounts of completed orders (e.g. completed_amounts) • Correct index-based traversal of aligned lists	10

5	Total and average calculations	• A function that computes totals (e.g. calculate_total) • A function that computes averages (e.g. calculate_average) • Validation for empty lists using len()	12
6	Highest completed order detection	• Variable storing max value (e.g. max_amount) • Correct loop-based comparison logic • Retrieval of associated order_id and email via index alignment	10
7	Email domain analysis	• Counters for gmail vs non-gmail (e.g. gmail_count, other_count) • Correct use of string condition checks	8
8	Suspicious order detection	• List storing warning messages (e.g. flags or warnings) • Conditional logic using HIGH_VALUE_THRESHOLD and status checks • Properly formatted warning strings	12
9	Final business report	• A multi-line string report variable (e.g. report) • Inclusion of all required metrics • Clear formatting using \n and f-strings	10

10	User-driven status query	• Variable storing user input (e.g. chosen_status) • Validation against VALID_STATUSES • Accumulator variable for status total (e.g. status_total)	6
TOTAL			100